# Implementation

*Membri del gruppo (Gruppo-03):*
- Francesco Lemmo - 0612705508 (Group leader)
- Leonardo Luongo - 0612706039
- Francesco Monda - 0612705277
- Mattia Vitali - 0612706221

## Note implementative

Rispetto a quanto riportato nei documenti precedenti, durante la fase di implementazione e per andare incontro ad alcune richieste funzionali dell'interfaccia grafica, sono stati modificati alcuni comportamenti del sistema.
L'inserimento degli operatori "+" e "-" avviene per mezzo del bottone riservato. Per eseguire l'operazione, a differenza delle altre operazioni, è necessario inserire l'operatore all'interno della *TextArea* e in seguito eseguirla premendo "Enter".
Un comportamento simile è assunto anche dalle operazioni sulle variabili, infatti, una volta selezionata l'operazione da eseguire ed inserita la variabile interessata, si procede ad eseguirla premendo anche qui il tasto "Enter".

## Differenze rispetto al design

L'implementazione ha portato, come già detto, ad alcune modifiche ed aggiunte rigurado ai metodi delle classi.
Analizziamo in maniera sintetica queste modifiche:

*ComplexNumber*
- `public boolean equals(Object object):` per il confronto di due numeri complessi;
- `private static boolean isNegativeZero(double value):` siccome il tipo double prevede lo zero sia negativo che positivo, questo metodo statico permette di riconoscere ciò e restituire *true* se "value = -0.0" altrimenti restituisce *false.*

*StackManager*
- `public int getSize():` restituisce la dimensione dello stack;
- `public ListIterator<ComplexNumber> getIterator():` restituisce null nel caso lo stack sia vuoto, altrimenti restituisce un ListIterator per la visita dello stack (si utilizza questo tipo di iteratore poiché ci permette di visitare lo stack in ordine inverso).

*VariableManager*
- `public Set<Map.Entry<String, Variable>> getVariablesEntry():` restituisce un *set di entry* della collezione delle variabili.

*InputValidator*
- *CorrectInputFormat (InnerClass per formattare l'input in forma corretta)*
  - `public CorrectInputFormat(String input):` costruttore;
  - `public String toString():` restituisce l'input come una stringa in formato corretto;
  - `private void tidyUpTheInput(String input):` svolge una valutazione preliminare sulla correttezza della stringa, eliminando caratteri superflui.
- `private void instanceOperations():` inserisce la coppia operazione-codiceOperazione all'interno di una mappa.

- `public boolean isComplexNumber(CorrectInputFormat correctInput)`: verifica che l'input sia un numero complesso;
- `private static int countOccurrences(String source, String target)`: metodo statico che restituisce il numero di occorrenze di un stringa all'interno di un'altra.

*Calculator*

- `public ComplexNumber getLastResult()`: restituisce l'ultimo elemento inserito nello stack;
- `public Set<Map.Entry<String, Variable>> getVariablesEntry()`: restituisce un *set di entry* della collezione delle variabili all'interno di VariableManager;
- `public ListIterator<ComplexNumber> getStackIterator()`: restituisce un ListIterator per la visita dello stack all'interno di StackManager.

## Eccezioni implementate

Tutte le eccezioni sono non-controllate e quindi estendono RuntimeException

- `NotEnoughElementsException`: viene lanciata se nello stack non sono presenti sufficienti elementi per eseguire un'operazione;
- `StackIsEmptyException`: estende NotEnoughElementsException e viene lanciata se lo stack è vuoto;
- `StackIsFullException`: viene lanciata se lo stack è pieno;
- `VariableIsEmptyException`: viene lanciata se si tenta di eseguire un'operazione che richiede l'acquisizione di un valore di una variabile non ancora inizializzata.

## Tabella di Tracciabilità

La tabella di tracciabilità è stata aggiornata in relazione all'implemetazione delle funzionalità richieste. Le funzionalità sono implementate nei file *java* indicati.

| # | Requirement | Design | Implementation | Testing |
|---|---|---|---|---|
| 1.0 | **Stack data structure** | StackManager | StackManager.java | |
| 2.0 | **Input number(s) or input operation** | Calculator | Calculator.java | |
| 3.0 | **Text Area** | UserInterface | Calculator_v2.fxml | |
| 4.0 | **Complex numbers** | ComplexNumber | ComplexNumber.java | |
| 5.0 | **Operazioni aritmetiche:** | Calculator | Calculator.java | |
| 5.1 | **Addizione** | Calculator | Calculator.java | |
| 5.2 | **Differenza** | Calculator | Calculator.java | |
| 5.3 | **Moltiplicazione** | Calculator | Calculator.java | |
| 5.4 | **Divisione** | Calculator | Calculator.java | |
| 5.5 | **Radice quadrata** | Calculator | Calculator.java | |
| 5.6 | **Inversione del segno** | Calculator | Calculator.java | |
| 6.0 | **Flusso di esecuzione delle operazioni** | Calculator | Calculator.java | |
| 7.0 | **Operazioni sullo stack:** | StackManager | StackManager.java | |
| 7.1 | **CLEAR** | StackManager | StackManager.java | |
| 7.2 | **DROP** | StackManager | StackManager.java | |
| 7.3 | **DUP** | StackManager | StackManager.java | |
| 7.4 | **SWAP** | StackManager | StackManager.java | |
| 7.5 | **OVER** | StackManager | StackManager.java | |
| 8.0 | **26 variabili** | VariablesManager | VariableManager.java | |
| 9.0 | **Operazioni sulle variabili:** | Calculator | Calculator.java | |
| 9.1 | **Salva nella variabile (>x)** | Calculator | Calculator.java | |
| 9.2 | **Copia nello stack (<x)** | Calculator | Calculator.java | |
| 9.3 | **Addizione (+x)** | Calculator | Calculator.java | |
| 9.4 | **Sottrazione (-x)** | Calculator | Calculator.java | |
| 10.0 | **Intefaccia utente** | UserInterface | Calculator_v2.fxml | |

## Analisi dei ritadi

I ritardi delle consegne sono stati provocati principalmente dalla difficile organizzazione al di fuori degli orari universitari ed una nuova sfida come quella dello sviluppo di un'applicazione con un'interfaccia grafica funzionale, user friendly ed esteticamente piacevole.

-F.L., L.L., F.M, M.V.